Teachers

Simone Viani
Software Architect R&D
email: simone.viani@sinfo-one.it

Luca Zaccomer
Analyst Developer R&D
email: luca.zaccomer@sinfo-one.it

Internet Protocols

TCP / IP

HTTP / HTTPS

TCP / IP

Connection oriented

Each connection identified by two pairs:

```
server-host:port + client-host:port
```

usually static

usually dynamic

Tools

- `ping`
- `netstat | ss`
- `nslookup | host | dig`
- `telnet`
- `curl | wget`
- `nc | netcat | socat`
- `ssh`
- `ip addr / ip route`

HTTP / HTTPS

Stateless protocol

- Each request is independent to the previous
- Socket connections can be closed and reopened at any time
- Clients can open parallel connections
- Cannot identify client by connection
- Can identify client session by headers
  - Cookies
  - Bearer token

HTTP / HTTPS

Request
- URL
- Method
- Headers
- Body

Universal Resource Location

GET POST PUT DELETE HEAD OPTIONS …

Cookies, Authentication, Tokens, Metadata, Correlation-ID, …

Response
- Status
- Headers
- Body

1xx – Info/continue
2xx - Success
3xx - Redirection
4xx – Client Error
5xx – Server Error

HTTP Message Example

Request

```
GET /private/login.html HTTP/1.1
Host: www.example.org
```

Response

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 295

<html>
  <head>
    <title>Login</title>
  </head>
  <body>
    <form method="POST" action="/private/dologin.jsp">
        User <input name="username"/>
        Password <input name="password" type="password"/>
        <input type="submit" value="Login">
    </form>
  </body>
</html>
```

HTTP Message Example

Request

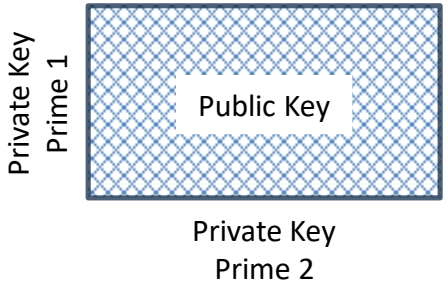
```
POST /private/dologin.jsp HTTP/1.1
Host: www.example.org
Content-Type: application/x-www-form-urlencoded
Content-Length: 30

username=alice&password=secret
```

Response

```
HTTP/1.1 303 See Other
Location: /private/welcome.html
```

TLS



Public - Private key pairs

Sender encrypt message with receiver public key
Receiver decrypt message with its private key

Keys generated and signed by a Certificate Authority (CA)

Global Catalog of Root CA
Hierarchical Certificate Chain

TLS Certificates

Certificates declare purposes
- Domain Name
- Code Signing

Server Certificates
- Self-signed
- Private CA
- Flexible support

Client certificates (optional)
- Identifies the client account

Root CA, Private CA, development

Corporate Proxy/Firewall, HTTPS Inspection

Import private CA, Ignore expiry date, Ignore subject-domain mismatch

TLS Tools

Let’s Encrypt

A nonprofit Certificate Authority providing TLS

Certificate Chain

acme.sh

An ACME protocol client written purely in Shell (Unix shell) language

Microservices

The Twelve Factors

Two Hard Things

Architecture
- Stateless / Stateful
- Synchronous / Asynchronous
- Cache / Persistence
- Admin / Monitoring

Communication
- SOAP
- REST / Websocket
- gRPC

SOAP Web Service

W3C Standard
Dispatch messages to remote objects

WSDL – Service definition
XML – Content payload

Language Mappings
- JAX-WS : Java Object <-> Web service
- JAXB : Java Object <-> XML content

RESTful Web Service

Exploit the HTTP standard!

Richardson Maturity Model
- Level 1 : Resources
- Level 2 : Verbs
- Level 3 : HATEOAS (resource linking)

Often the message body is in JSON format

Tools
- Postman

JSON by Examples

```
null
```

```
true
```

```
123.76
```

```
"hello"
```

```
[ false, 456, "abc" ]
```

```
{ "id": 789, "name": "xyz", "active": true }
```

```
[
  { "id": 101, "name": "abc", "description": null },
  { "id": 102, "name": "def", "selected": true },
  { "id": 103, "name": "xyz", "enabled": false }
]
```

GraphQL

Developed by Facebook

Server defines metadata
- Convert requests to SQL (or other) queries

Client defines queries and mutations
- Scalable, efficient, powerful client

Security

Principle of least privilege
Zero trust security model

DO
- Validate / sanitize client input
- Choose a framework, ie: Spring

DO NOT
- Expose plain identifiers
- Reinvent the wheel

Security OWASP

OWASP Top 10

1. Broken Object Level Authorization
2. Broken Authentication
3. Broken Object Property Level Authorization
4. Unrestricted Resource Consumption
5. Broken Function Level Authorization
6. Unrestricted Access to Sensitive Business Flows
7. Server Side Request Forgery
8. Security Misconfiguration
9. Improper Inventory Management
10. Unsafe Consumption of APIs

Basic Web Technologies

HTML
CSS
Javascript
AJAX
WebSocket
Web Server / Proxy / Reverse Proxy

HTML

W3C Standard
Markup language for semantic description of documents.
Browser download page and resources from a web server.

HTML 4 -> HTML 5
MDN doc

What Web Can Do Today

CSS

W3C Standard
Stylesheet language for layouts, geometry, borders, colors, effects, animations, …

MDN Doc

AJAX

Use Javascript to make asynchronous requests to the server and incrementally update the page

Evolution:
- XMLHttpRequest
- Fetch API

WebSocket

MDN Doc

Persistent HTTP connection for fast, bidirectional communication.
HTTP Header to upgrade protocol.

STOMP

Subscribe to topics, send/receive messages.

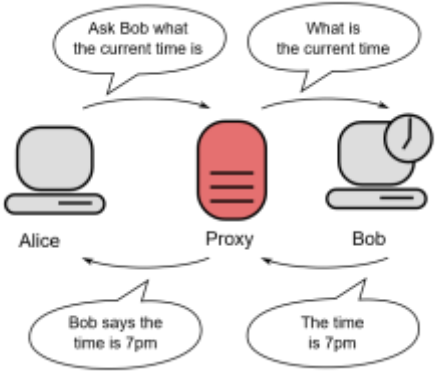
Web Server

Software that can receive and reply to HTTP messages.

Alternatives:
- Apache httpd
- Ngnix
- Envoy Proxy

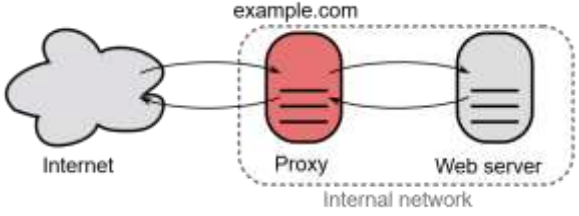
Proxy / Reverse Proxy

Proxy
- Intermediate between a client and a web server
- Manage cache, security, privacy
- A web server can act as a proxy



Forward Proxy
- From internal to external network

Reverse Proxy (RPX)
- From external to internal network
- Support Name Based Virtual Hosts



RPX + NBVH + TLS

SNI - Server Name Indication

- Support Name Based Virtual Hosts with TLS
- One IP can serve multiple services by hostname
- Plain text value of the hostname to contact

Advanced Web Technologies

JWT
- Token to authorize a client in a stateless server

WebRTC
- Real-time communication

WebAssembly
- Binary format for browser or server
- Interop with Javascript

CORS
- Enable multiple backend domains
- Enforced by browsers

API Gateway
- Enable multiple backend services
- Apply security policies
- Monitoring and analytics

Load Balancer
- Enable multiple service instances
- Monitoring

Javascript

ECMAScript implementation
Interpreted, Duck Typing language
Executed in Web Browsers
Executed in Operating Systems, ie: NodeJS

Evolution:
- ES5 - classic
- ES6 - ECMAScript 2015
- Typescript

Javascript Environments

Browser API:
- Can access DOM elements (HTML tags)
- Security limitations applies
- Event-driven, Single threaded

Operating System API - NodeJS:
- Can use processes
- Can access filesystems
- Can access I/O

Javascript Data Types

boolean
- `true, false`

number
- `decimal value, NaN, Infinity`
- `MAX_SAFE_INTEGER = 2^53 – 1`

string
- `'abc’, "abc"`

object
- `{ }`
- `{ id: 1, name: 'abc’ }`

function
- `function identity(x) { return x }`
- `const identity = (x) => x`

undefined
- `not initialized`
- `absent value`

null
- `intentional absent object`

Javascript Objects

Objects have a prototype
- Can have dynamic properties (associative array)

Functions are first-class objects
- Can be input to functions
- Can return functions (higher order function)

Javascript Declarations

var
- Declares a variable, optionally initializing it to a value
- Prefer const or let.

let
- Declares a block-scoped, local variable, optionally initializing it to a value

const
- Declares a block-scoped, read-only constant

Javascript Destructuring Assignment

```
const obj1 = { a: 1, b: 2, c: 3 }
const { a, b } = obj1
console.assert(a === 1)
console.assert(b === 2)

const obj2 = { a: 10, b: 20, c: 30 }
const { a: a2, ...obj2Rest } = obj2
console.assert(a2 === 10)
console.assert(obj2Rest.b === 20)
console.assert(obj2Rest.c === 30)

const obj3 = { ...obj2, b: 22 }
console.assert(obj3.a === 10)
console.assert(obj3.b === 22)
console.assert(obj3.c === 30)
```

```
const propName = 'a'
const {
    [propName]: propValue
} = obj1
console.assert(propValue === 1)
```

Javascript Destructuring Assignment

```
const arr1 = [ a, a2, 100 ]
const [ e1, e2 ] = arr1
console.assert(e1 === arr1[0])
console.assert(e1 === 1)
console.assert(e2 === arr1[1])
console.assert(e2 === 10)

const arr2 = [ 0, ...arr1, 1000 ]
console.assert(arr2.length === 5)
```

```
const [ , , ...arr3 ] = arr2
console.assert(arr3.length === 3)
console.assert(arr3[0] === 10)
console.assert(arr3[1] === 100)
console.assert(arr3[2] === 1000)
```

Javascript Functions

```
function fa(x) {
    function fb(y) {
        function fc(z) {
            return x + y + z
        }
        return fc(3)
    }
    return fb(2)
}
console.assert(fa(1) === 6)
```

Javascript Promise

Promise: asynchronus operation
- then, catch, finally, callbacks
- new keywords: async / await

Switch context on asynchronus operation

JsDoc

Inline documentation for Javascript sources

- Supported by popular editors
- Can declare types and properties
- Can be used as an alternative to Typescript

Functional Programming

Data / Function separation
- ie: MVC

Data Immutability
- Constant values: Object.freeze()
- Copy-on-write: Immer, Immutable

Pure Function
- Idempotence: result only depends on inputs
- No side-effects: never changes their inputs
- State transition: copy-on-write input to result

Reactive Programming

Declarative paradigm
Handle stream of asynchronous events

Examples:
Redux
- Store, subscribe, dispatch
- Minimal features only, easy learning

RxJs
- Subject, subscribe, next
- Full featured, more complexity

Classic Web Application

Server-Side Rendering (SSR)

- Dynamic HTML pages created by the server
- Merge templates with data
- AJAX to update part of a page
- SEO friendly
- JSP, ASP.NET, …

Single Page Application

Client-Side Rendering

- Only a minimal single HTML page
- Dinamically update page with Javascript
- Virtual routing of locations
- Not SEO friendly

Progressive Web App

What PWA Can Do Today

App-like user experience
Manifest
Web Workers

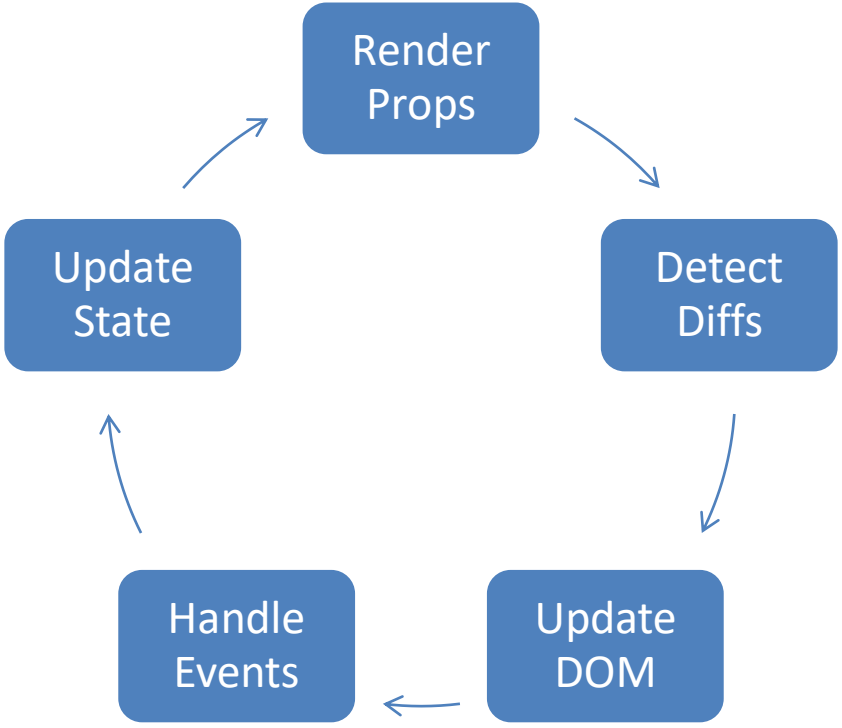
Rendering Libraries

React
- Render components in a virtual DOM
- Minimal update of real DOM elements

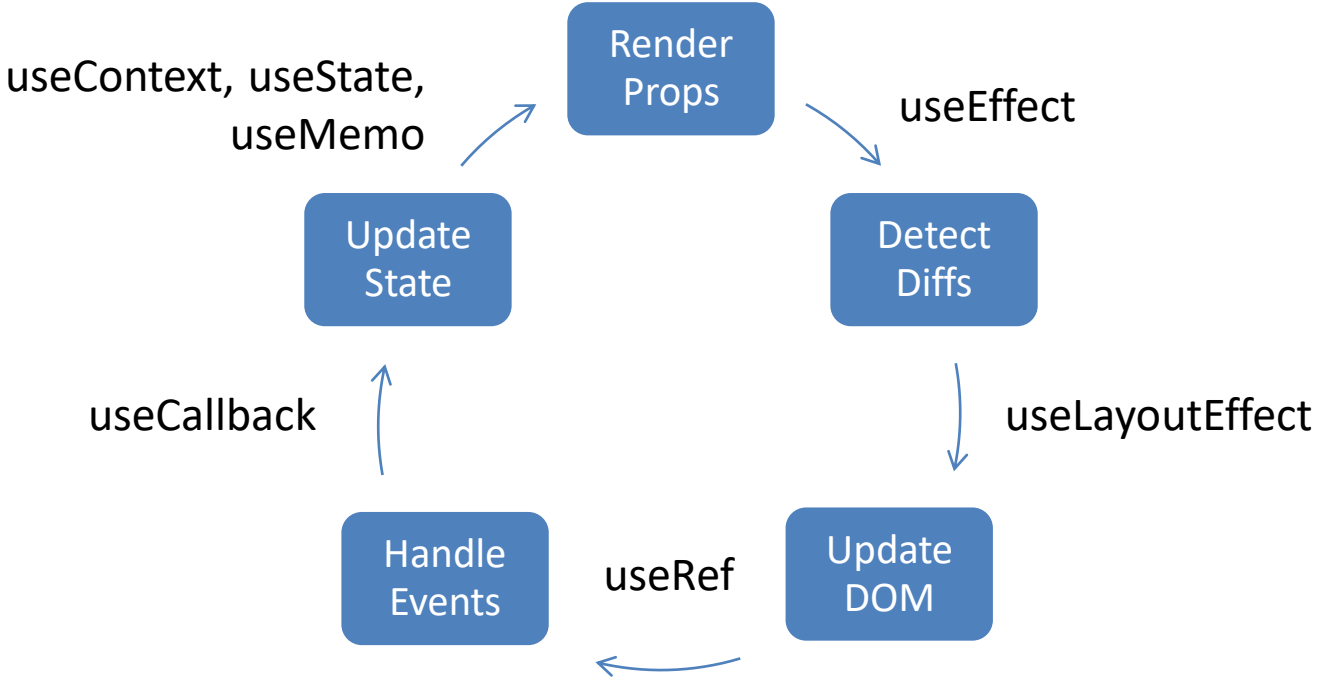
React Redux
- Store: single immutable state object
- Dispatch function: send actions to the store
- Reducer function: handle actions and return the next state

React Router
- Navigation between virtual pages

React Render Cycle



React Hooks Lifecycle



Routing

Location: current browser address

History: keep previous locations
- get, push, replace functions

Route: an available match for locations
- supports path variables

Match: data for a matched route
- resolved path variables

Development Tools

Runtime: NodeJS
Package Manager: NPM
Build System: Nx

Transpiler: Babel, TSC, SWC
Bundler: Webpack, Rollup, Parcel
Linting: ESLint
Testing: Jest, Cypress
Dev Server: Express

All in one: Bun

Source Version Control: Git
Monorepo: Git submodules

Rendering: ReactJS

Quality Assurance: Sonarqube